

08. Product Requirements Document (PRD)

QuantPipeline — private investment decision-intelligence platform for medium-term portfolio decisions

Document purpose	Define the product in a way that engineering, UX, validation, and operations can build against without guessing
Target user	Single sophisticated owner-operator
Decision horizon	Weeks to months
Primary product type	Decision-support and portfolio-intelligence application, not a retail trading broker
Companion docs	Research pack, technology recommendation, UX appendix, and forthcoming engineering specifications

This PRD defines the product requirements for QuantPipeline, a private decision-intelligence platform built to support medium-term equity investment decisions. The system is intended to synthesize multiple evidence layers — market/news text, social sentiment, technical analysis, portfolio context, machine-learning signals, reinforcement-learning-informed reasoning, and operational health — into a coherent recommendation framework.

The PRD is intentionally explicit because the product is not a simple dashboard. It is a multi-layer system whose success depends on product coherence. The UI, data model, analytical engines, validation methods, and operational reliability model must all reinforce the same core promise: help the user make better medium-term decisions with clear reasoning, visible risk, and honest confidence signals.

The product is for private use, but private use does not reduce the need for rigor. On the contrary, because the product will influence real investment decisions and serve as the user's own analytical cockpit, it must be clear, replayable, operationally honest, and methodologically disciplined.

Target Architecture

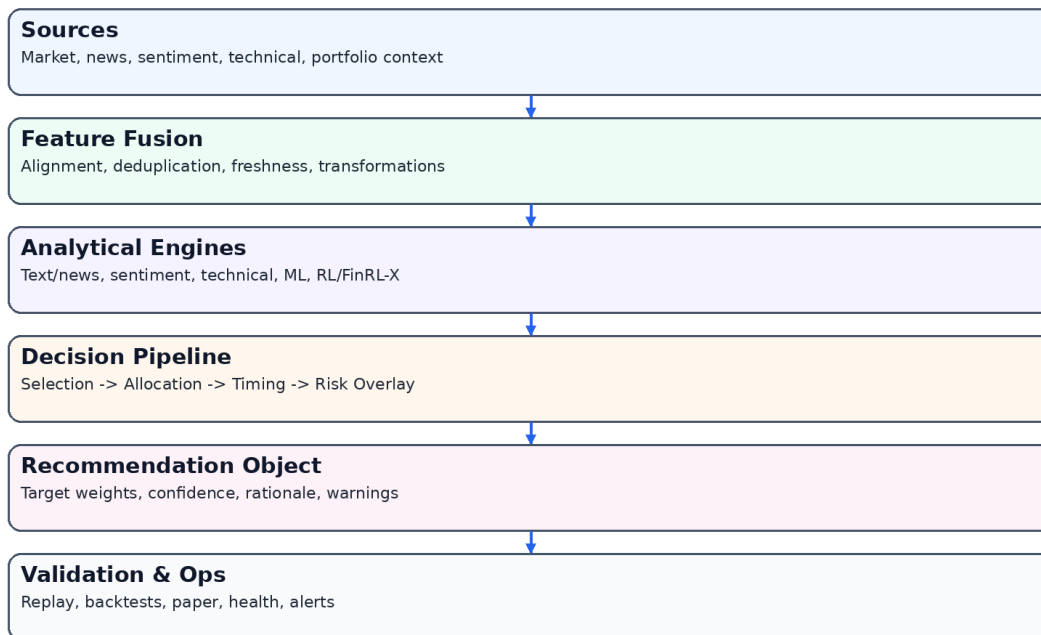


Figure 1. High-level product architecture.

Site Information Architecture

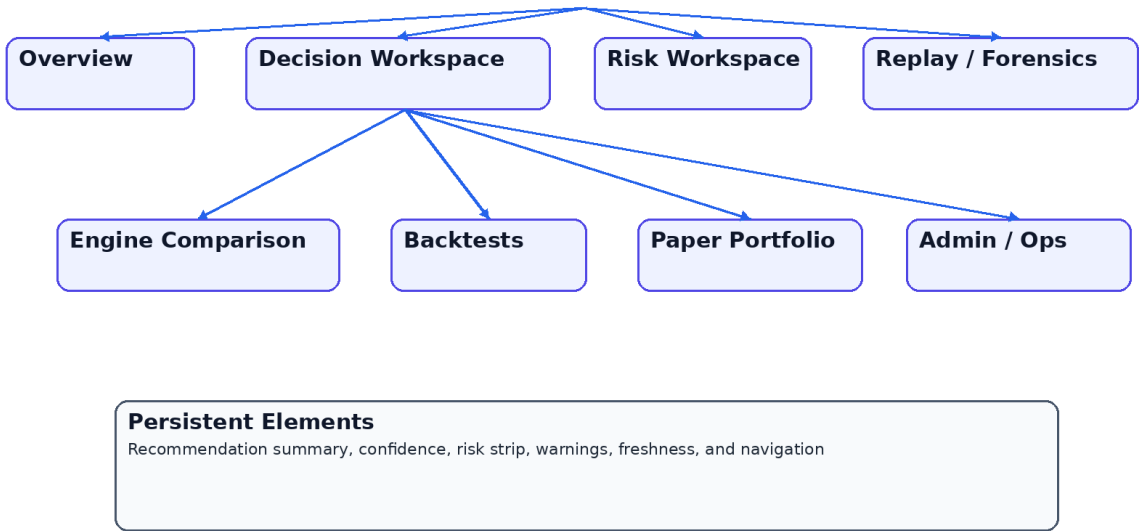
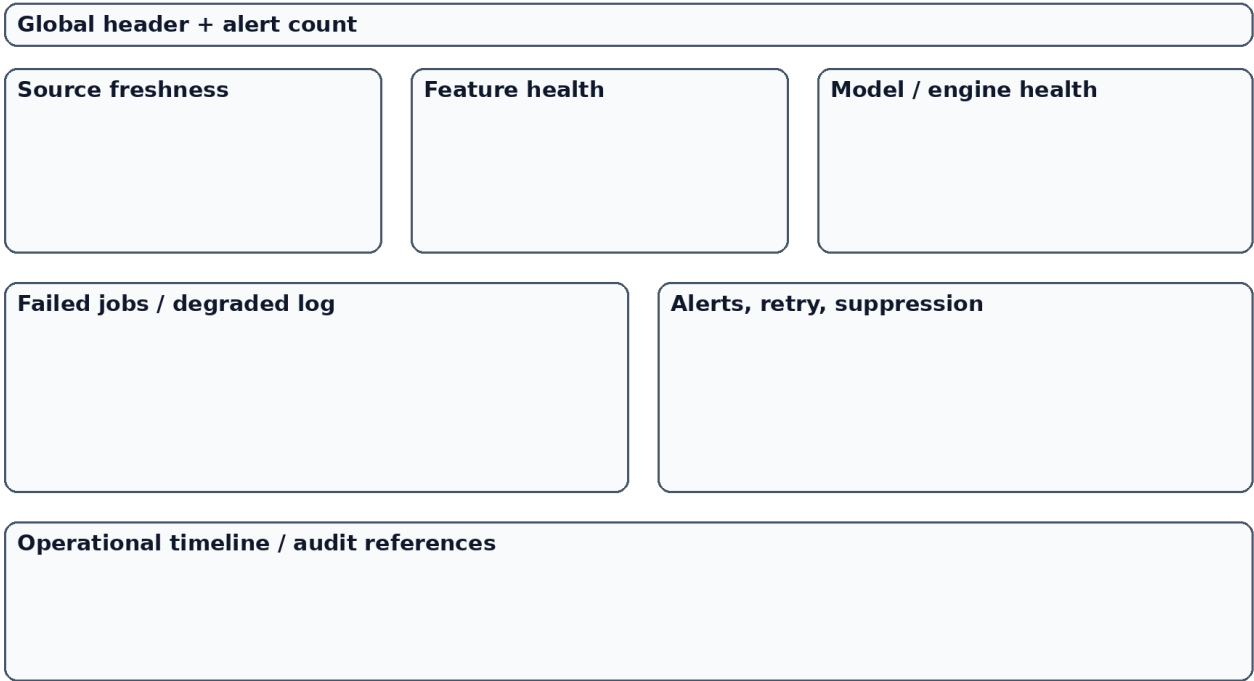


Figure 2. High-level information architecture and top-level navigation concept.

Admin / Ops Workspace



1. Product Vision

QuantPipeline should become the user's primary environment for turning fragmented market evidence into disciplined portfolio decisions. The product should not behave like a generic market portal, a broker interface, or a collection of disconnected analytics widgets. It should behave like one integrated decision system.

That means every major surface in the product must answer a practical question: what is the current recommendation, why does the system believe it, how strong is that belief, what are the main risks, what changed recently, and what should the user review before acting.

The vision is therefore not just to surface data, but to reduce cognitive fragmentation while preserving enough transparency for trust.

These statements should influence implementation choices directly. The product should optimize for medium-term decision usefulness, transparent synthesis, and repeatable review rather than for feature theatrics or disconnected data abundance.

2. Product Goals

- Provide a unified recommendation-centric dashboard for medium-term investment decisions.
- Combine multiple analytical modalities without forcing the user to manually stitch them together.
- Expose trust, confidence, disagreement, and freshness directly in the product.
- Represent portfolio context and risk as first-class product objects.
- Support replay, review, and post-hoc learning from prior recommendations and outcomes.
- Maintain operational honesty through visible source, engine, and publication states.

3. Non-Goals

1. The first release is not a brokerage execution platform.
2. The first release is not a public social investing network.
3. The first release is not a generalized quant research IDE for arbitrary strategy coding.
4. The first release is not a data-vendor replacement or financial terminal clone.
5. The first release is not an institutional OMS/EMS.

4. Target User and Operating Context

The target user is a sophisticated single owner-operator who wants a serious analytical environment, not a simplified retail toy. The user can handle depth but should not be forced to navigate avoidable clutter.

The operating rhythm is medium-term. Signals and recommendations should therefore be optimized for decisions held over weeks or months, not for seconds, scalping, or ultra-high-frequency execution.

This context changes many product decisions. It favors synthesis over speed theater, policy clarity over hyperactivity, and replayable conviction over endless flickering inputs.

5. Core Jobs to Be Done

Job	Desired outcome
-----	-----------------

Understand the current recommendation state	Know what the system currently prefers and why
Evaluate confidence and disagreement	See whether the evidence is aligned or conflicted
Review portfolio impact	Understand concentration, diversification, and risk implications
Track what changed	See when and why the recommendation moved
Inspect history and replay	Review prior recommendation states and outcomes
Validate methodology	Trust that the product measures itself honestly
Administer the system	Monitor freshness, failures, warnings, and suppressions

Jobs-to-be-done framing matters because the product is not merely informational. Each core surface should correspond to a real analytical job and reduce mental stitching work for the user.

6. Product Principles

- Recommendation-first, not widget-first
- Trust by exposure, not by opacity
- Portfolio-aware, not ticker-isolated
- Desktop-primary but mobile-credible
- Operationally honest
- Replayable and auditable
- Progressive disclosure instead of terminal clutter

These principles should govern all design and engineering trade-offs. A superficially advanced product can still fail if its primary surfaces do not reveal trust, risk, change, and operational state clearly.

7. Product Scope Overview

Product area	In scope
Dashboard and overview	Yes
Recommendation object	Yes
News / text analysis	Yes
Social sentiment	Yes
Technical analysis	Yes
Portfolio and risk intelligence	Yes
Replay and historical review	Yes
Validation / backtests / paper	Yes
Admin and ops	Yes
Live brokerage execution	No for first release

8. Core Product Surfaces

The product should include a clear set of top-level surfaces rather than a loose collection of pages. At minimum, these surfaces should include an Overview or Home dashboard, the main decision Dashboard, Risk, Replay, Backtests / Validation, Paper Portfolio, Admin / Ops, and Settings or Controls.

The top-level navigation should reflect conceptual jobs, not implementation layers.

The product shell should support continuity between surfaces so that the user can move from recommendation to risk, replay, and operational state without losing context.

A practical implication of this area is that the interface and backend contracts should be shaped together. The system should not bolt product meaning on top of raw engine outputs after the fact.

9. Canonical Recommendation Object

The product requires a canonical recommendation object. This is one of the most important requirements in the entire PRD.

The recommendation object should include current stance, trust or confidence expression, main supporting evidence, main contradicting evidence, relevant portfolio implications, risk flags, freshness information, and links into deeper diagnostics.

The recommendation object should be designed as a product object, not merely as a backend payload.

10. Dashboard Experience

The main dashboard should be the center of gravity of the product. It should answer the most important questions in one coherent surface: what does the system currently prefer, what changed, what is the confidence structure, what are the top risks, and what actions or reviews are most relevant right now.

The dashboard must balance density and legibility. Dense does not mean cluttered.

The dashboard should support both quick review and deeper drill-down.

11. News and Text Intelligence

The system should ingest and interpret market-relevant news and text.

The requirement is not merely to show headlines, but to surface the parts of text intelligence that change decision quality: directionality, relevance, novelty, source quality, thematic concentration, and narrative shift.

If news materially changed the stance or confidence of a recommendation, the product should say so explicitly.

12. Social Sentiment Intelligence

Social-sentiment analysis should be treated as one evidence family among several, not as a magical truth source.

The product should indicate whether social sentiment is supportive, contradictory, extreme, unstable, or low-quality.

Social sentiment should enrich context and sometimes add timing value, but it should remain visibly bounded within the evidence stack.

13. Technical Analysis Layer

Technical analysis should provide structure, regime awareness, timing context, and momentum / trend interpretation in a way that is aligned with a medium-term holding horizon.

The product should surface a clear technical summary, key chart context, and the main technical support or contradiction to the current recommendation.

This area should be tightly curated in product design; the requirement is decision support, not chart maximalism.

14. Portfolio and Risk Layer

Portfolio and risk must be first-class product objects.

The product should include a dedicated Risk surface and also summarize portfolio implications on the main dashboard.

When a recommendation looks attractive but creates concentration or diversification problems, the product should make that tension visible rather than burying it.

15. Replay and Historical Review

Replay is essential. The user must be able to inspect prior recommendation states, understand what changed, and compare system belief evolution to subsequent market outcomes.

The replay experience should distinguish between recommendation changes driven by market movement, new source evidence, model output changes, and operational or policy changes.

Replay should be an explicit screen family rather than a hidden admin utility.

16. Validation, Backtests, and Paper

The product requires a validation surface that goes beyond static performance claims.

Backtests, validation snapshots, and paper-portfolio behavior should be visible so that the user can judge whether the system's method still deserves trust.

Paper portfolio workflows should connect recommendation logic to a plausible position-management abstraction without becoming a live brokerage terminal.

17. Admin and Ops

The product requires a serious Admin / Ops surface because the system depends on multiple sources, scheduled jobs, engine families, validations, and publication logic.

The Admin / Ops area should expose source freshness, job health, engine health, recommendation publication status, warning generation, suppressions, and notable reliability events.

This area is especially important because the product is private; the owner needs an honest cockpit for both user-facing decisions and system-facing reliability.

18. Global UX States

- Loading
- Empty but healthy
- Stale
- Degraded
- Conflicted
- Suppressed
- Warning / action required
- Healthy and current

Global states matter because a sophisticated analytical product often fails not in its happy path but in ambiguous conditions. The product must tell the truth when data is stale, signals conflict, or publication has been intentionally suppressed.

19. Success Metrics

Metric family	Example success question
Recommendation usefulness	Did the user get to a reviewable recommendation quickly and clearly?
Trust clarity	Could the user explain why confidence is high or low?
Risk awareness	Could the user see portfolio implications before acting?
Replay value	Could the user understand what changed and learn from it?
Operational transparency	Could the user see when the system was degraded or stale?
Interaction efficiency	Could the user perform key review tasks without friction?

Success metrics should not be limited to performance numbers. Product quality must also be judged by clarity, trust calibration, operational honesty, and the user's ability to detect when not to trust the current output.

20. Release Strategy

6. Foundation release: canonical recommendation object, dashboard shell, risk summary, admin health basics, and replay skeleton.
7. Analytical depth release: stronger news/text, social sentiment, technical synthesis, and richer portfolio context.
8. Validation release: backtests, methodology surfaces, paper portfolio, and deeper replay.
9. Hardening release: operational reliability, governance, suppressions, alerts, and QA maturity.

Release sequencing should follow product coherence, not raw feature temptation. The first release should already feel like one product.

Release sequencing should follow product coherence, not raw feature temptation. The first release should already feel like one product.

Release sequencing should follow product coherence, not raw feature temptation. The first release should already feel like one product.

Release sequencing should follow product coherence, not raw feature temptation. The first release should already feel like one product.

21. Risks and Failure Modes

- Widget sprawl without canonical synthesis
- Overloaded dashboard that hides the decision
- Invisible reliability problems
- False confidence from weak or stale evidence
- Validation disconnected from live product behavior
- Portfolio risk treated as a side module
- Mobile parity pursued in the wrong places

These risks should be read as product risks, not just engineering risks. For example, false confidence is a product failure even if the code executed correctly.

22. Dependency Map

Dependency	Why it matters
Stable recommendation schema	Needed for dashboard, replay, validation, and admin consistency
Source provenance	Needed for trust and replay
Portfolio state model	Needed for risk-aware recommendations
Health/status model	Needed for operational honesty
Historical persistence	Needed for replay and validation
UX design system	Needed to keep complexity legible

23. Acceptance at Product Level

At product level, the system should pass only if a user can land on the dashboard, understand the current recommendation and its confidence structure, review the main risk implications, recognize whether the system is healthy, and navigate to deeper evidence or replay without confusion.

24. Open Questions Reserved for Engineering Specs

- Exact schema of the recommendation object
- Exact event model and telemetry taxonomy
- Exact API endpoints and payload fields
- Exact persistence strategy and storage technologies
- Exact validation methodology details
- Exact admin control privileges and intervention mechanics

These questions are intentionally delegated to the engineering documents that follow. The PRD defines the product truth those documents must implement without violating.

25. Final PRD Statement

QuantPipeline should be built as a coherent private decision-intelligence product whose center of gravity is the recommendation object and its surrounding trust, risk, replay, validation, and operational context. Every later document should be judged by one standard: does it help implement this product cleanly, transparently, and without forcing the user to perform the synthesis manually?

26. Detailed User Scenarios

Scenario A: Morning market review. The user opens the dashboard before the trading day, checks whether the system recommendation changed overnight, scans the confidence structure, sees that news support strengthened but technical posture weakened, and decides to defer any action until a risk screen review confirms concentration impact.

Scenario B: Mid-week narrative shift. The user notices that social sentiment spiked unusually while core news relevance is mixed. The product should make clear whether this is a true supportive shift, a low-quality crowd reaction, or a warning-worthy divergence that should not materially change the recommendation yet.

Scenario C: Portfolio reallocation. The user evaluates whether an attractive recommendation is still acceptable given existing exposure. The product should show current concentration, benchmark-relative posture, and whether the recommendation pushes the portfolio into an uncomfortably narrow thematic shape.

Scenario D: Reliability incident. A source becomes stale or a scheduled job fails. The user should see a visible degraded or stale state, understand which evidence layers are affected, and know whether recommendation publication was suppressed or only confidence was reduced.

Scenario E: Historical learning. The user enters replay mode to understand a recommendation that looked compelling three weeks ago but underperformed. The product should reveal what the system believed then, what changed later, and whether the miss was due to weak data, a regime break, portfolio constraints, or policy logic.

The product implication of this section is that engineering, UX, and validation should all treat it as executable intent. These are not decorative paragraphs. They are meant to reduce ambiguity, prevent drift, and ensure that future implementation decisions remain anchored in the product's actual purpose.

27. Detailed User Stories

As a user, I want to understand the current recommendation in one screen so that I can decide whether to drill deeper or act later.

As a user, I want to see why confidence is high or low so that I do not confuse signal volume with actual conviction.

As a user, I want to understand whether different evidence families agree or disagree so that I can interpret stability versus fragility.

As a user, I want to see whether a recommendation is stale or degraded so that I do not act on outdated intelligence.

As a user, I want to understand portfolio impact so that I can judge whether a good signal still creates a bad portfolio trade-off.

As a user, I want a replay surface so that I can learn from past recommendation changes and strengthen my process over time.

As a user, I want validation and paper surfaces linked to the live product so that I can test whether current methodology still deserves trust.

As a user, I want an admin and ops cockpit so that I can diagnose reliability issues without leaving the product context.

28. Prioritized Requirement Buckets

Must-have: canonical recommendation object, dashboard shell, trust/confidence expression, portfolio-aware risk summary, replay skeleton, source freshness visibility, basic admin health, and stable navigation architecture.

Should-have: disagreement decomposition by evidence family, narrative change summaries, richer chart interactions, paper portfolio review workflows, validation summaries embedded into live product surfaces, and strong mobile review flows.

Could-have later: watchlists, scenario notes, external notifications, richer custom controls, configurable strategy lenses, and more advanced comparison workspaces.

Out-of-scope early: live order routing, public sharing, social features, and institutional workflow management.

29. Edge Cases and Failure UX Requirements

When no recommendation is publishable, the product should say so explicitly and explain whether the reason is data insufficiency, conflict, stale state, validation gating, or a manual/admin suppression condition.

When one evidence family is missing, the system should not silently flatten the confidence model. It should show partial operation and explain its consequences.

When portfolio data is unavailable, the dashboard should degrade honestly and prevent risk-dependent claims that would otherwise overstate certainty.

When replay data is incomplete, the product should show which historical layers are missing rather than pretending continuity exists.

When the mobile layout cannot support a deep desktop interaction, the mobile UX should redirect to a simplified path rather than presenting a broken or cramped equivalent.

30. Product Decisions That Must Stay Central

The recommendation object remains the nucleus of the product.

Risk and portfolio context remain first-class rather than subordinate.

Operational truth remains visible on primary surfaces when relevant.

Replay and validation remain part of product trust, not side documents.

Mobile remains important, but not at the cost of destroying desktop clarity.

Admin and ops remain part of the product ecosystem, not an afterthought.

31. Product Requirement Matrix

Area	Requirement	Why it matters	Acceptance signal
Dashboard	Must surface current recommendation and confidence in top fold	Primary job completion	User can explain recommendation after first load
Risk	Must summarize portfolio implications on dashboard and dedicated risk page	Avoid isolated ticker decisions	User can state risk trade-offs before acting
Replay	Must store and display prior recommendation states	Supports learning and trust	User can compare now vs prior state
Ops	Must expose health and freshness	Supports operational honesty	User can detect degraded system state
Validation	Must connect methodology status to live product	Avoid false confidence	User can see whether validation trust is weakening
Mobile	Must support review, not forced parity	Preserve usability on phone	Critical review flows remain intact on mobile
Admin	Must provide source and job visibility	Supports maintenance and diagnosis	User can inspect root issue families

32. Product Readiness Review Questions

- Can the user understand the recommendation without assembling it manually?
- Can the user tell whether the system is healthy right now?
- Can the user see what changed and why?
- Can the user understand confidence structure rather than only score outputs?
- Can the user see whether the portfolio context weakens an otherwise attractive signal?
- Can the user inspect validation trust and replay history?
- Can the user recover from degraded conditions with the UI still making sense?

If the answer to any of these review questions is no, the product is not ready regardless of how many features were technically implemented. Readiness in this product is fundamentally about coherence, clarity, and honest synthesis.

33. Detailed Screen Intent by Surface

Overview screen: provide immediate orientation, health snapshot, current recommendation, and recent change summary in under one minute.

Dashboard screen: provide the richest cross-evidence synthesis and the best path to deeper analysis.

Risk screen: focus on portfolio-level interpretation, concentration, scenario sensitivity, and tension between attractive signals and prudent exposure.

Replay screen: provide historical state comparison, narrative change trail, and post-hoc diagnostic structure.

Validation screen: explain whether the live decision framework continues to deserve trust under observed data and market regimes.

Paper screen: show how recommendations would or did behave in a managed hypothetical portfolio context.

Admin / Ops screen: expose hidden truth about sources, jobs, health, suppressions, and operational interventions.

This section should remain active during future iterations. As features expand, these statements should be used as a filter to decide whether the roadmap is strengthening the product's core identity or merely increasing surface area without improving decision quality.

34. Product Language and Microcopy Direction

The product should speak in precise but not overly academic language. Confidence should not be described with theatrical certainty. Warnings should be concrete and attributable. Degraded states should explain what is affected and what is not.

Recommendation language should distinguish between directional preference, supporting evidence, contradicting evidence, and portfolio caution. The product should avoid phrases that imply guaranteed outcomes.

Microcopy must reinforce the product promise: organized judgment, explicit uncertainty, visible risk, and disciplined review.

35. Long-Term Product Evolution Guardrails

Do not let the product devolve into a generic market portal.

Do not let new widgets enter the dashboard unless they materially improve the recommendation workflow.

Do not let ops truth disappear behind engineering-only tooling.

Do not let validation become a side document disconnected from the live system.

Do not allow mobile work to distort the desktop control-center role of the dashboard.

Do not add execution complexity before the recommendation product is genuinely mature.

36. Product-Level Assumptions

The user values synthesis more than entertainment.

The user is willing to inspect evidence, but does not want to perform manual stitching.

The user wants real transparency about data and operational weakness.

The user prefers a strong desktop experience while still needing meaningful mobile review.

The user benefits from historical replay and method validation rather than purely present-tense dashboards.

The product can create more value by improving decision quality than by maximizing data quantity.

37. Comprehensive Product Checklist

- Canonical recommendation object defined and visible
- Confidence/trust expression clear and not hidden
- Disagreement between evidence families visible
- Risk implications shown before deep drill-down
- Replay available and connected to recommendation history
- Validation linked to live product trust
- Operational health visible when relevant
- Stale and degraded states clearly represented
- Mobile review path preserved
- Admin / ops not split into a completely disconnected product

A completed checklist item is not proof by existence alone. The implementation should satisfy the spirit of the requirement. For example, a replay page that exists but does not actually explain change is not a completed replay requirement. Likewise, a mobile page that technically loads but destroys information hierarchy is not a satisfactory mobile implementation.

38. Final Product Readiness Conclusion

The PRD should be considered satisfied only when QuantPipeline behaves like one integrated decision-support product rather than a stack of technical features. The recommendation should be visible, the trust structure should be understandable, risk should be portfolio-aware, replay and validation should support learning, and operational truth should remain visible. This standard is intentionally demanding because the product's value depends on disciplined coherence more than on feature volume.